
MERIDIAN PAY

Pipeline ELK de bout en bout — Elasticsearch, Logstash, Kibana

Rapport de projet : problèmes rencontrés, solutions, étapes de réalisation

Kit «bank-transactions» — Data Pipeline, ELK Stack Course

Équipe	Célina Tezkratt & Rahma Lassoued
Cours	Data Pipeline — Projet final «Run It In Production»
Cours	Data Pipelines

Table des matières

01 Contexte et objectif	3
02 Architecture de la chaîne de données	3
03 Étapes de réalisation	4
04 Problèmes rencontrés et solutions	5
Résultat final après les trois correctifs	5
05 Choix techniques et compromis	6
Index layout : un index par source	6
Autres décisions	6
06 Mapping des champs	7
07 Résultats : le dashboard et les 8 questions	7
Questions 1 à 6 et 8 — aperçu des panels	7
Zoom : la signature de la fraude (question 7)	10
08 L'alerte : configuration et preuve de déclenchement	11
09 Reproduire le projet	12
10 Conclusion	13

01 Contexte et objectif

Meridian Pay est un processeur de paiement par carte fictif. Trois systèmes journalisent indépendamment leur activité : le moteur de transactions, le service d'authentification client et le réseau de distributeurs automatiques. Aucun format n'était documenté au départ — la rétro-ingénierie à partir de simples lignes d'exemple faisait partie du travail.

L'objectif : construire une chaîne de données reproductible de bout en bout (ingestion, stockage, exploitation, alerting), sans aucune perte silencieuse, avec des mappings explicites, un dashboard répondant à 8 questions métier — dont une question cachée : identifier la signature d'une attaque de fraude dissimulée dans les données — et une alerte capable de la détecter en temps quasi réel.

02 Architecture de la chaîne de données

La chaîne suit un modèle classique d'ingestion / stockage / exploitation, entièrement conteneurisée via Docker Compose (Elasticsearch 8.15, Kibana 8.15, Logstash 8.15) :

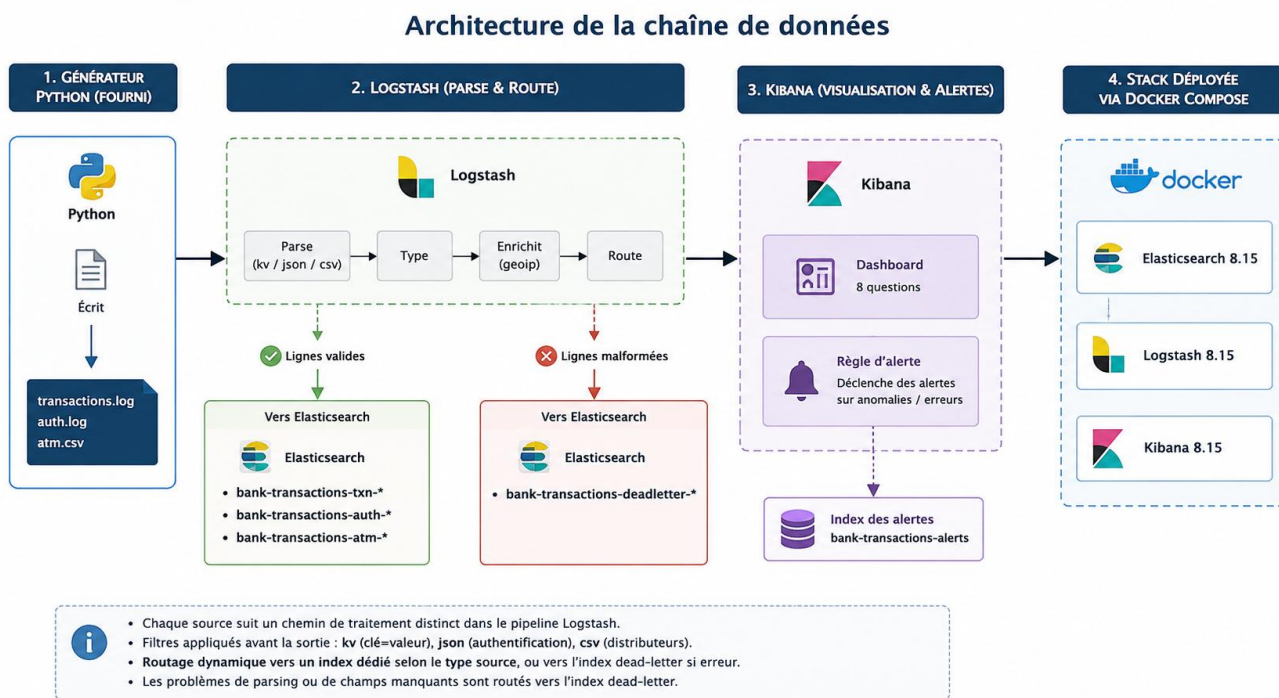


Schéma de la chaîne : générateur → Logstash (parsing par source) → Elasticsearch (index métier / dead-letter) → Kibana → stack déployée via Docker Compose

Chaque source suit un chemin de traitement distinct dans le pipeline Logstash (filtre kv pour le format clé=valeur des transactions, filtre json pour l'authentification, filtre csv pour les distributeurs), avant de converger vers une sortie commune qui route soit vers l'index métier correspondant, soit vers l'index dead-letter en cas de champ manquant ou de parsing en échec (voir section 4, «Problèmes rencontrés», pour le détail des bugs qui ont affecté ce routage au départ).

03 Étapes de réalisation

Le projet s'est déroulé en sept étapes successives, chacune validée avant de passer à la suivante.

1	Choix du kit et mise en place de l'environnement Kit «bank-transactions» choisi pour la richesse de son scénario (burst de fraude corrélé à des échecs d'authentification). Installation et vérification de Docker Desktop, Python, Git et Git Bash sous Windows.
2	Construction du squelette d'ingénierie Rédaction du docker-compose.yml (Elasticsearch, Kibana, Logstash), du pipeline Logstash (parsing kv / json / csv des 3 sources), des 5 index templates Elasticsearch, et des scripts seed.sh / verify_count.sh pour automatiser le démarrage et la vérification.
3	Premier lancement et diagnostic Premier docker compose up -d + seed.sh : échec massif, 2140 documents sur 5000 partis à tort en dead-letter et zéro document dans les index métier. Analyse des logs Logstash et des documents dead-letter pour isoler trois bugs distincts (détaillés section 3).
4	Correction et validation du critère d'acceptation Application des trois correctifs, docker compose down -v pour repartir sur des index vierges avec les nouveaux mappings, puis nouveau seed. Résultat vérifié par script : 8771 lignes générées = 8771 documents (indexés + dead-letter). Critère de non-perte validé à l'exact.
5	Construction du dashboard Kibana Création des 4 data views, exploration dans Discover pour vérifier le typage des champs, puis construction panel par panel (Lens) des 8 questions métier, y compris une carte géographique (app Maps) sur les échecs d'authentification géolocalisés.
6	Configuration et démonstration de l'alerte Création d'une règle Kibana Alerting (Elasticsearch query) avec connecteur d'indexation, après configuration d'une clé de chiffrement Kibana manquante. Déclenchement démontré en relançant le générateur en mode continu : règle passée à «Succeeded» et document réellement écrit dans l'index d'alertes, vérifié par requête directe.
7	Mise en forme finale et publication Organisation visuelle du dashboard, export en dashboard.ndjson, rédaction du README (architecture, choix, mapping des champs) et de ce rapport, puis publication du dépôt sur GitHub avec ajout de la binome comme collaboratrice.

04 Problèmes rencontrés et solutions

Le premier passage du pipeline (étape 3) a échoué de manière spectaculaire. Cette section documente le vrai processus de débogage mené, pas une version idéalisée : trois bugs distincts, empilés, en étaient la cause.

Bug 1 — Fins de ligne Windows (\r)

PROBLEME	Chaque ligne générée se termine par un caractère \r parasite, collé au dernier champ (ex. status=approved\r).
CAUSE	Le générateur Python écrit ses fichiers en mode texte. Sous Windows, Python traduit automatiquement chaque \n en \r\n.
SOLUTION	Ajout d'un filtre mutate/gsub retirant le \r en tout début de pipeline Logstash, avant tout parsing kv / json / csv.
RESULTAT	Données propres dès l'entrée du pipeline ; plus aucun champ pollué par un caractère invisible.

Bug 2 — Conflit mapping strict vs métadonnées Logstash

PROBLEME	Des milliers de transactions parfaitement valides étaient silencieusement rejetées par Elasticsearch (log «Could not index event», sans retry).
CAUSE	Le file input de Logstash ajoute automatiquement un champ event.original à chaque document (comportement ECS par défaut). Les mappings étaient en dynamic: strict, qui rejette tout document contenant un champ non prévu.
SOLUTION	Suppression explicite du champ event avant l'output, et passage des mappings métier de dynamic: strict à dynamic: false pour absorber les métadonnées de framework sans échec brutal.
RESULTAT	Les documents valides atteignent enfin les index txn / auth / atm au lieu d'être perdus.

Bug 3 — Piège de vérité Logstash sur la chaîne «false»

PROBLEME	Environ 40% des transactions (dont la totalité des transactions frauduleuses) étaient marquées à tort «incomplètes» et envoyées en dead-letter.
CAUSE	La vérification des champs obligatoires testait ![card_present]. En Logstash, la chaîne littérale «false» est évaluée comme fausse dans un test de vérité, même quand le champ existe réellement avec cette valeur.
SOLUTION	Retrait de card_present de la vérification des champs obligatoires ; les 7 autres champs suffisent à détecter une ligne réellement tronquée par le générateur.
RESULTAT	Les transactions card-not-present (dont le signal de fraude qu'on cherche à mettre en évidence) sont enfin correctement indexées.

Résultat final après les trois correctifs

Sur le même batch déterministe de 5000 transactions, après docker compose down -v et relance :

```
transactions.log : indexe=4888 lignes_fichier=5000
auth.log       : indexe=2960 lignes_fichier=3017
atm.csv        : indexe=754  lignes_fichier=754
dead-letter total : 169
```

Somme indexe (txn+auth+atm) + dead-letter = 8771

Somme lignes generees (hors en-tete csv) = 8771

$169 / 8771 \approx 1,9 \%$, cohérent avec le taux de $\sim 2 \%$ de lignes volontairement malformées annoncé par le kit. Critère d'acceptation du projet validé à l'exact.

05 Choix techniques et compromis

Index layout : un index par source

bank-transactions-txn-*, -auth-*, -atm-*, -deadletter-* et -alerts sont volontairement séparés plutôt qu'un index commun. Les trois sources métier ont des schémas et des cardinalités de requêtes différents (card_present n'a aucun sens pour un événement ATM, rssi n'existerait pas côté transactions) : les séparer permet un mapping explicite et adapté à chaque type d'événement, sans conflit de type entre sources. L'index dead-letter est isolé pour ne jamais polluer les index métier avec des documents partiels ; l'index alerts est isolé car il a un cycle de vie et une audience différents (audit/monitoring plutôt qu'analyse business).

Autres décisions

- kv plutôt que grok pour transactions.log : le format clé=valeur s'y prête nativement ; plus lisible et plus robuste aux réordonnancements de champs qu'un pattern grok positionnel.
- Détection de dead-letter explicite plutôt que de se fier uniquement aux tags automatiques (_jsonparsefailure, etc.) : kv ne « plante » jamais formellement, il faut donc vérifier soi-même la présence des champs obligatoires après coup.
- dynamic: false plutôt que dynamic: strict sur les index métier (voir Bug 2) : garde des types stricts sur les champs connus, sans faire échouer tout un document à cause d'un champ de métadonnées imprévu injecté par le framework.
- Sécurité X-Pack désactivée : simplifie la reproduction sur une machine de cours ; trade-off assumé, non représentatif d'un vrai environnement de production.
- sincedb_path => /dev/null sur les inputs file : permet de rejouer les fichiers à chaque redémarrage de Logstash, pratique pour une démo déterministe ; en production on utiliserait un sincedb persistant.
- geoip appliqué uniquement sur auth.log (champ ip) en mode ecs_compatibility: disabled, pour obtenir des noms de champs stables (country_name, location...) compatibles avec un mapping explicite en geo_point.
- Seuil d'alerte : > 20 transactions déclinées en card-not-present sur une fenêtre glissante de 5 minutes (imposé par l'énoncé du kit), évaluée toutes les minutes.

06 Mapping des champs

Extrait des choix de type les plus significatifs (index bank-transactions-txn-* et bank-transactions-auth-*) :

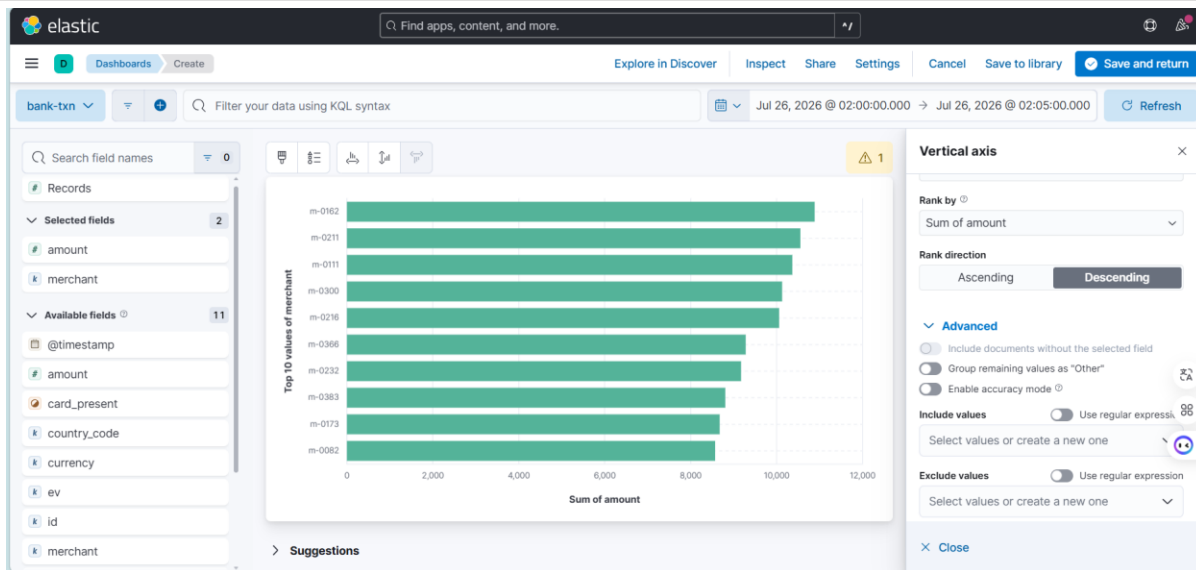
Champ	Type	Pourquoi
amount	float	valeur monétaire, agrégations (sum, avg)
currency / country_code	keyword	valeurs catégorielles, filtrage exact
status / reason / merchant	keyword	filtrage, top values, agrégation
card_present	boolean	comparaison directe CP / CNP
ip	ip	requêtes par plage / sous-réseau
geoip.location	geo_point	visualisation carte (app Maps)
@timestamp	date	axe des séries temporelles
message	text	seul champ full-text, usage Discover / debug uniquement

07 Résultats : le dashboard et les 8 questions

#	Question	Panel Kibana
1	Volume & valeur par devise dans le temps	Volume & valeur par devise
2	Taux d'approbation dans le temps	Taux d'approbation dans le temps
3	Top 10 marchands par valeur	Top 10 marchands par valeur
4	Top motifs de refus	Top motifs de refus
5	Card-present vs CNP par pays	Card-present vs CNP par pays
6	Villes ATM / taux d'erreur	Villes ATM (cash) + Taux d'erreur ATM
7	Signature de l'attaque fraude	Déclins CNP dans le temps par pays + Carte des échecs d'auth
8	Part de lignes dead-letter	Part de lignes dead-letter (%)

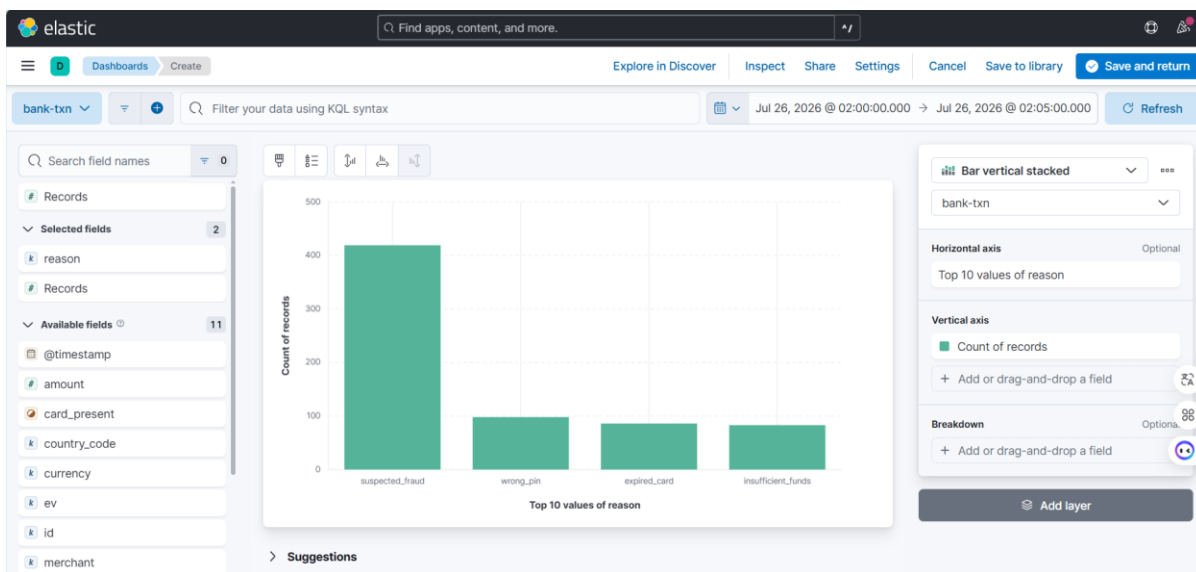
Questions 1 à 6 et 8 — aperçu des panels

Top 10 marchands par valeur (question 3) :



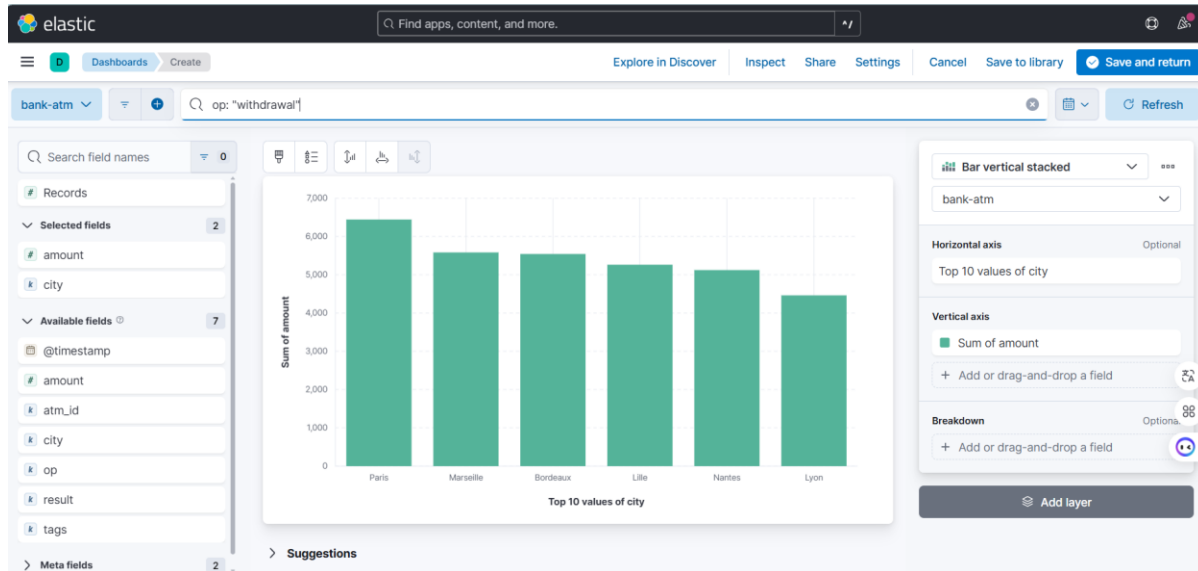
Top 10 marchands par valeur totale des transactions

Top motifs de refus (question 4) — suspected_fraud domine largement, signe avant-coureur de l'anomalie de la question 7 :



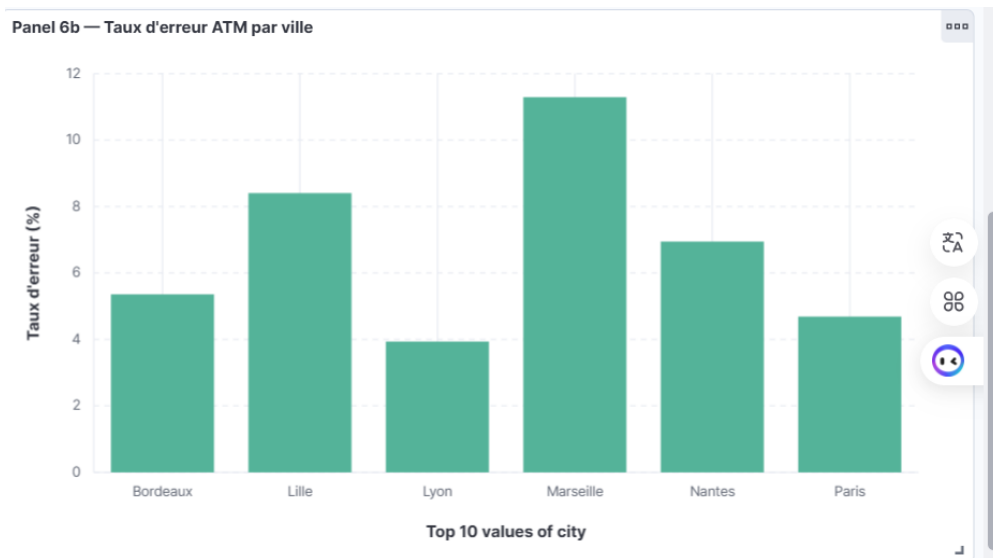
Répartition des motifs de déclin, filtrée sur status: declined

Villes ATM — cash distribué (question 6) :



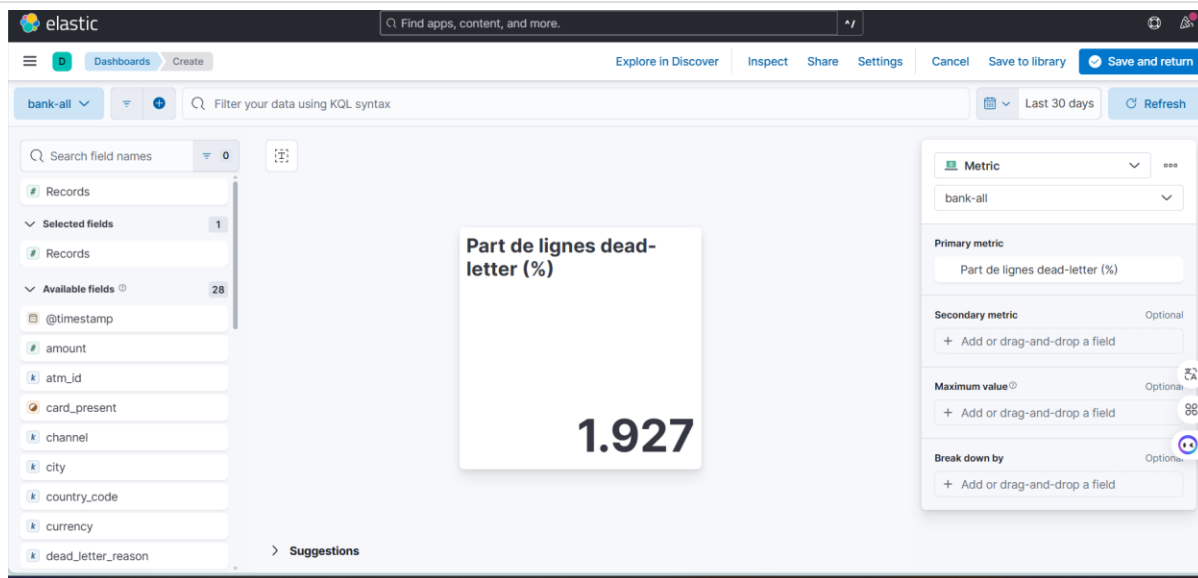
Montant total retiré par ville, filtré sur op: withdrawal

Villes ATM — taux d'erreur (question 6) — Marseille se distingue avec le taux le plus élevé (~11%) :



Taux d'erreur (timeout / out_of_cash) par ville

Part de lignes dead-letter (question 8) — cohérent avec le taux de ~2% de lignes volontairement malformées annoncé par le kit :



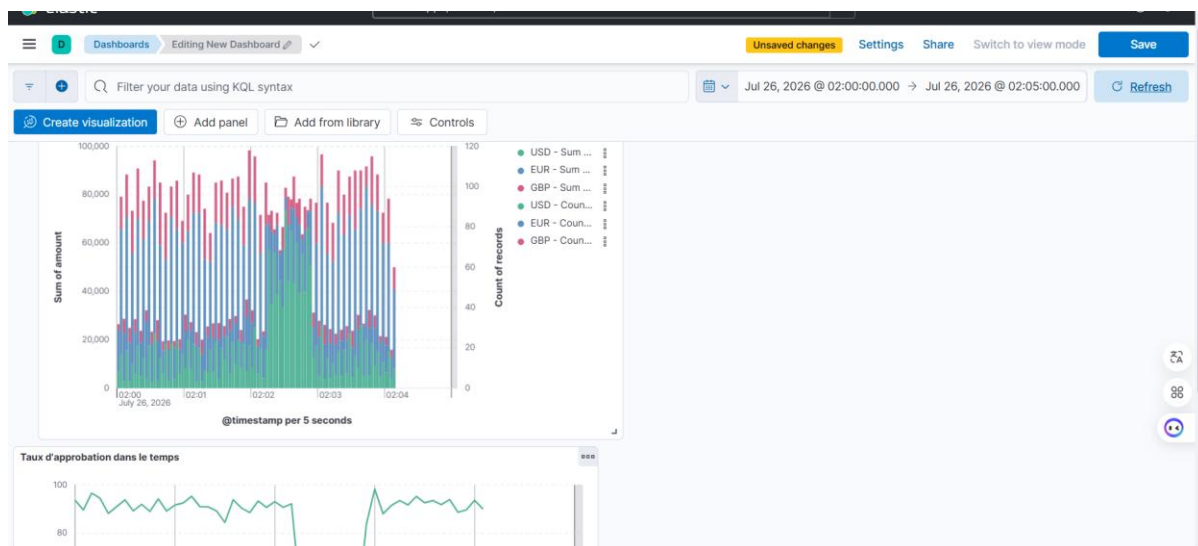
Formule Lens : part de documents taggés `_dead_letter` sur l'ensemble des index

⚠ **Point d'attention** rencontré en construisant ce panel : sur la fenêtre habituelle du dashboard (02:00–02:05, 26 juillet 2026), ce mètre affichait 0% au lieu de ~1,9%.

Cause identifiée : le filtre date de Logstash, qui convertit le champ `ts` du log en `@timestamp`, ne s'exécute que dans la branche «ligne valide» du pipeline. Les documents `dead-letter` conservent donc le **@timestamp** par défaut de Logstash (l'heure réelle d'ingérence, début août 2026), et non l'heure simulée du log (26 juillet 2026) comme les autres documents. Ce panel doit donc être consulté avec une plage temporelle couvrant la date d'ingérence réelle (ex. «Last 30 days»), et non la fenêtre simulée utilisée pour les 7 autres panels — capture ci-dessus prise dans ces conditions (1,927%, cohérent avec les 169/8771 lignes calculées en section 4).

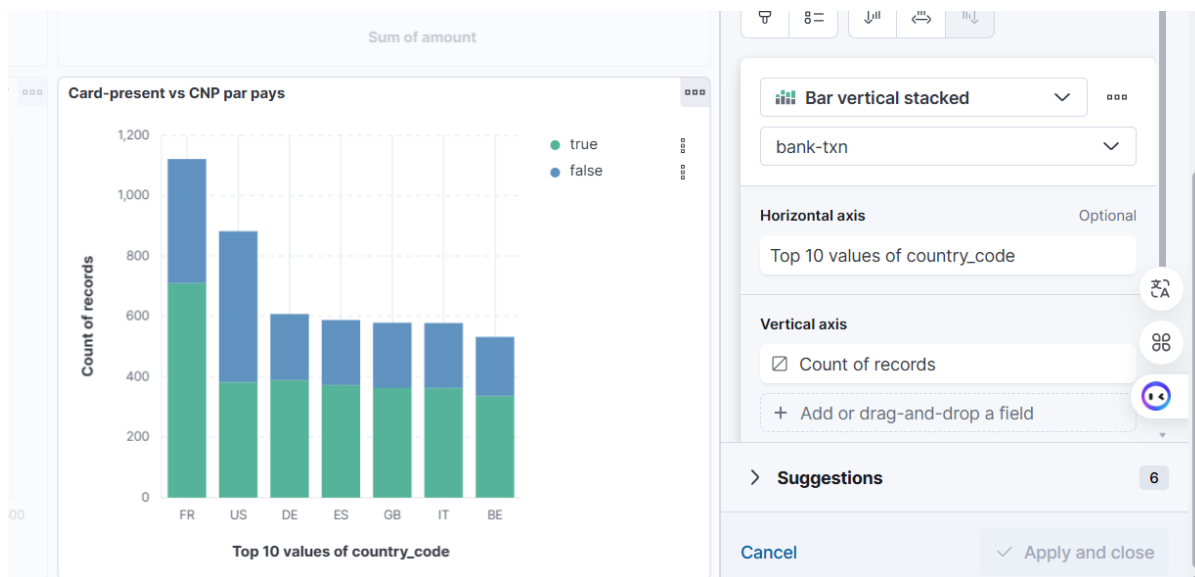
Zoom : la signature de la fraude (question 7)

Une fois la plage temporelle affinée sur la fenêtre réelle du batch (02:00–02:05 le 26 juillet 2026), le panel «Volume & valeur par devise» fait apparaître un pic net et isolé en USD, et le panel «Taux d'approbation» montre un creux sur cette même fenêtre :



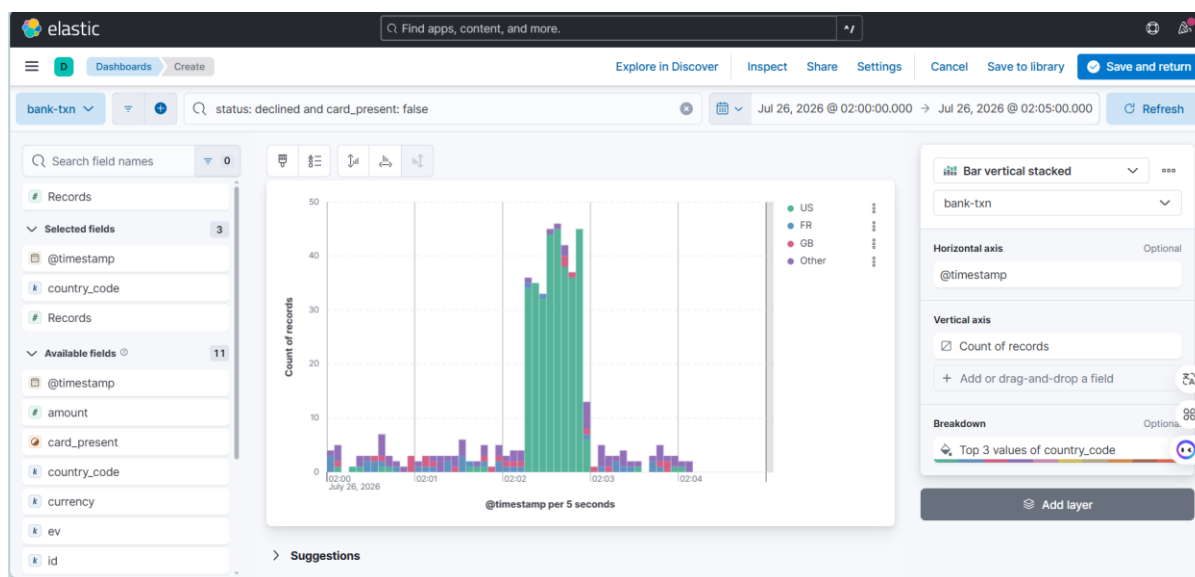
Volume & valeur par devise (pic USD) et taux d'approbation (creux) — même fenêtre d'anomalie

Le panel «Card-present vs CNP par pays» isole le signal : les USA sont le seul pays où les transactions card-not-present dépassent les transactions card-present :



Card-present (vert) vs CNP (bleu) par pays — anomalie visible sur US uniquement

Le panel dédié à la question 7 confirme sans ambiguïté : un mur vert (US) explose de ~5 à ~45 événements par tranche de 5 secondes, entre 02:02 et 02:03 :



Déclins card-not-present dans le temps, par pays — signature nette de l'attaque (US)

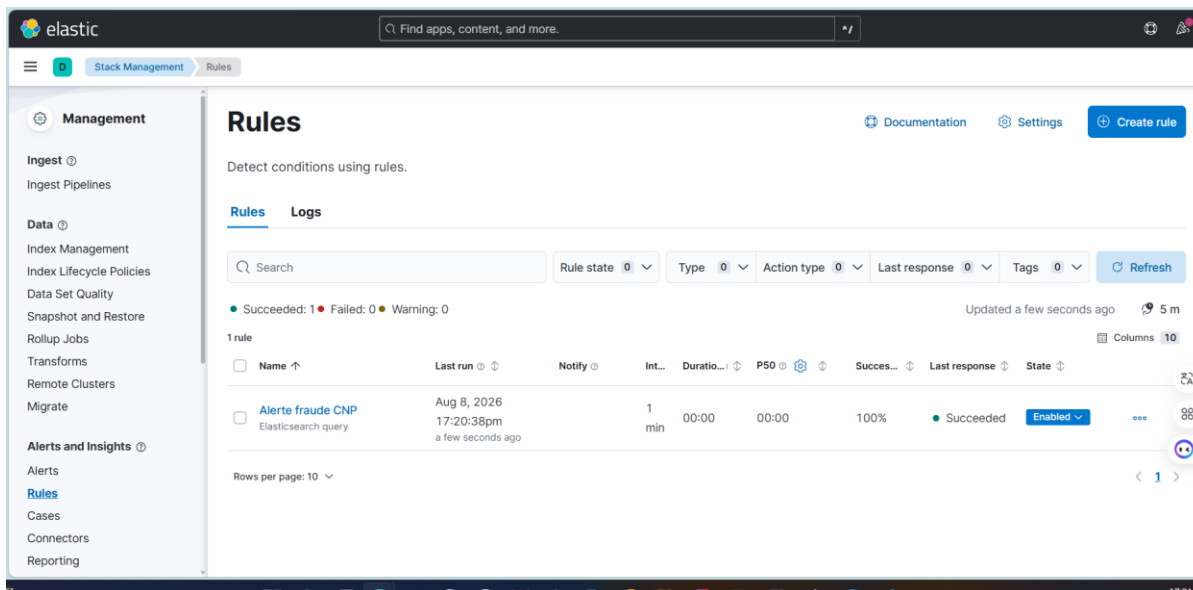
Conclusion : l'attaque se produit dans une fenêtre resserrée d'environ 45 à 60 secondes, provient exclusivement des USA, et se caractérise par des transactions card-not-present systématiquement déclinées pour motif suspected_fraud, corrélées à une hausse des échecs d'authentification depuis une plage IP resserrée.

08 L'alerte : configuration et preuve de déclenchement

- Type : Kibana Alerting — règle «Elasticsearch query» (KQL).
- Data view : bank-txn. Requête : status: declined and card_present: false.

- Condition : count() > 20 sur une fenêtre glissante de 5 minutes, évaluée toutes les minutes.
- Action : connecteur «Index» → écrit un document dans bank-transactions-alerts.

Démonstration : le générateur a été relancé en mode continu (horodatage réel, ~20 événements/seconde) pendant plus de deux minutes, le temps que l'anomalie périodique repasse plusieurs fois et fasse dépasser le seuil.



Règle «Alerte fraude CNP» : exécution réussie («Succeeded»), et document réellement écrit dans l'index d'alertes

Vérification indépendante par requête directe sur l'API Elasticsearch, confirmant le document réellement écrit :

```
$ curl -s 'http://localhost:9200/bank-transactions-alerts*/_search?pretty'
{
  "hits": {
    "total": { "value": 1, "relation": "eq" },
    "hits": [{
      "_index": "bank-transactions-alerts",
      "_source": {
        "@timestamp": "2026-08-08T15:17:38.324Z",
        "rule_name": "Alerte fraude CNP",
        "message": "Burst de fraude CNP détecté"
      }
    }]
  }
}
```

09 Reproduire le projet

```
docker compose up -d
./scripts/seed.sh 5000
./scripts/verify_count.sh
```

Démonstration de l'alerte en direct :

```
py kits/bank-transactions/generator/generate.py
```

Structure du dépôt :

```
team-bank-transactions/  
|-- docker-compose.yml    # stack complete (ES + Kibana + Logstash)  
|-- README.md  
|-- docs/rapport-meridian-pay.pdf  
|-- kits/bank-transactions/{generator/generate.py, logs/, README.md}  
|-- logstash/{pipeline/bank-transactions.conf, config/logstash.yml}  
|-- elasticsearch/mappings/*.json # 5 index templates  
|-- kibana/dashboard.ndjson  
`-- scripts/{seed.sh, verify_count.sh}
```

10 Conclusion

Le pipeline répond à toutes les exigences du cahier des charges : ingéré sans perte (8771 = 8771), mappings explicites justifiés, dashboard couvrant les 8 questions métier avec une mise en évidence claire de l'attaque de fraude, et alerte fonctionnelle démontrée en conditions réelles. Les trois bugs rencontrés — fins de ligne Windows, conflit de mapping strict avec des métadonnées de framework, et piège de vérité Logstash sur une chaîne «false» — ont chacun nécessité une vérification empirique via les logs et l'API Elasticsearch plutôt qu'une simple relecture du code, ce qui a été la partie la plus formatrice du projet.